ORIGINS / BROWSER TRUST / SECURE API DESIGN

CORS Explained

Understand browser trust, prove real data exposure, and build secure cross-origin APIs

---

A COMPLETE VISUAL LEARNING KIT

Understand the browser before the vulnerability.
Predict the request. Prove the read. Repair the contract.
Practice with fictional accounts on your own computer.

Haroon Shahid

First edition / Version 1.0 / 25 October 2026

Before you begin

CORS Explained

Understand browser trust, prove real data exposure, and build secure cross-origin APIs

By Haroon Shahid. First edition, version 1.0. 25 October 2026.

Copyright 2026 Haroon Shahid. The purchased publication is licensed for personal use under the included LICENSE.txt. The separately identified lab code is MIT licensed. The free preview may be shared unchanged with attribution. No resale or public redistribution of the full paid publication is granted.

This edition teaches responsible browser-security analysis through fictional, local examples. It does not certify professional competence. Framework and protocol chapters are version-sensitive review patterns. Read the Validation Report for executed test scope.

PDF code may wrap for page fit. The offline HTML and source files retain original code lines. The HTML reading edition also provides responsive text and native answer controls. On small screens, pan its diagrams and tables horizontally; no full PDF accessibility conformance is claimed.

Free preview: introduction, roadmap, and Chapters 1-3. The full package also includes all 59 chapters, answered reviews, offline HTML, practice workbooks, the local lab, and reference sheets.

Contents

Before you begin 2

Contents 3

Introduction 4

Learning roadmap 6

01. The web is a system of messages 7

02. HTML, JavaScript, and the first API call 9

03. HTTP requests, representations, and effects 11

Sources and further reading 14

Introduction

A page says the request failed. The user's nickname still changed.

A different page shows Access-Control-Allow-Origin: *. The data is a public timetable, shared on purpose.

A third page has a perfectly valid CORS response. It gives an unrelated website the signed-in user's private record.

All three cases can produce confusing screenshots. They have different causes, different impacts, and different repairs. This book teaches you to tell them apart.

What you will learn to do

You will learn how a browser identifies a source, decides which requests can leave, attaches eligible authority, and filters the response available to page code. Then you will learn to investigate a suspected failure, prove the smallest useful claim, repair the reader contract, and preserve legitimate use.

The book starts before CORS. It explains messages, URLs, JavaScript, cookies, origins, sites, and the Same-Origin Policy. You do not need to know those ideas in advance. Later chapters keep connecting the difficult details to those first foundations.

The recurring story

Atlas Ledger is a fictional records application. Avery Demo is its invented reader. The application has an API, one intended browser reader, and unwanted peer pages. Its private canaries are random teaching strings, not real secrets. We return to the same system so you can spend your attention on the changed boundary instead of learning a new story every chapter.

The local Origin Observatory lab makes selected cases observable. It shows results in the page and records secret-free server events. Some topics, including managed clouds, WebRTC, WebSockets, and public-to-private network permission, are architecture reviews rather than executed lab services. The Validation Report makes that scope explicit.

How the pages work

Each chapter has a goal, a plain definition, an analogy, original diagrams, worked reasoning, misconceptions, a practice prompt, a summary, and three answered checks. The six-panel browser trace is a teaching model of rules; the browser does not have human beliefs. A panel labelled accepts as evidence means a specific platform input is used, not that a person or application is trustworthy.

Predict before you test. Write down dispatch, authority, server result, and page-readable result separately. When you are wrong, find the first assumption that failed. That habit is more useful than memorizing a long list of header combinations.

Choose a reading route

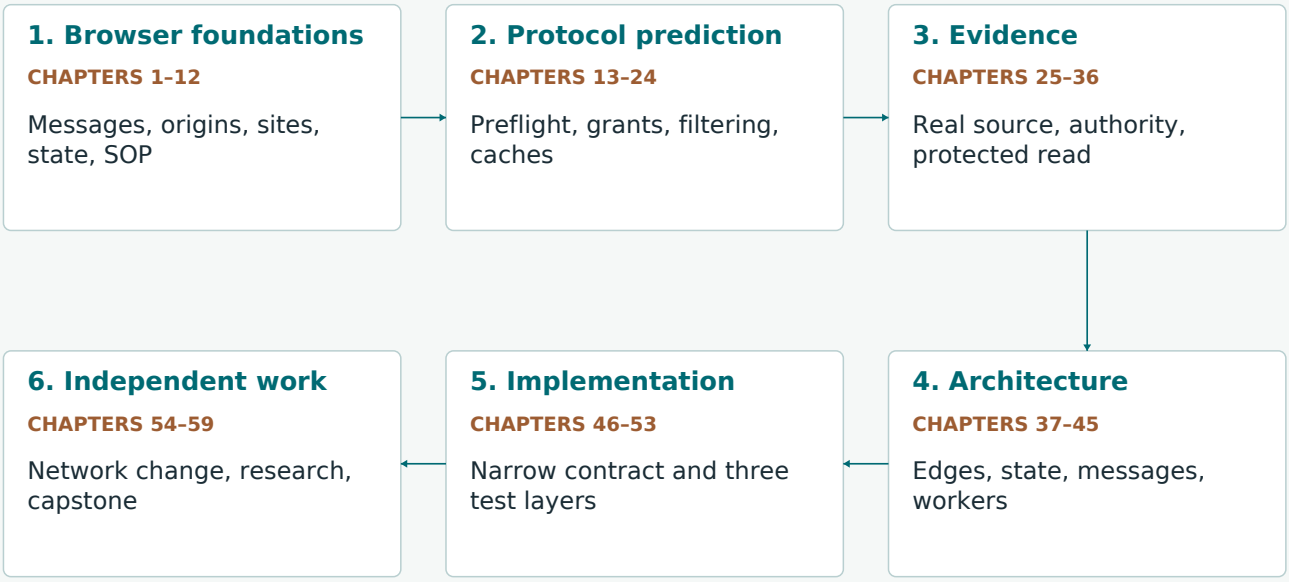
Start at chapter 1 if web security is new. If you build APIs, use the roadmap to locate the protocol and implementation chapters, then return to any foundation you cannot explain in plain language. If you assess applications, learn the evidence method before using a tool's risk label.

Use only systems you own or have explicit permission to assess. The runnable demonstrations are fixed to local fictional services. A useful proof shows the minimum protected information needed and never requires collecting another person's real records.

What this edition promises

It offers a coherent learning path, original visual explanations, answered practice, reproducible local demonstrations, and a dated primary-source register. It does not promise professional expertise from reading alone, exhaustive coverage of every future engine, or guaranteed findings. Its practical aim is that you can explain a new case, test its conditions, and justify its repair without guessing from a console message.

Concept dependency map: earn each next step



Learning roadmap

The dependency path

Messages and code lead to URLs. URLs lead to origins and sites. Those identities lead to the Same-Origin Policy and credential rules. Only then does CORS become a useful, deliberate response-sharing exception.

Part	Chapters	Ability to demonstrate
Understand the browser	1–12	Explain sources, state, resources, isolation, and usable authority
Follow the protocol	13–24	Predict dispatch, grants, filtering, caches, and redirects
Find and prove failures	25–36	Establish browser capability, private result, limits, and repair
Review the architecture	37–45	Trace proxies, clouds, state policy, messages, workers, and isolation
Implement the contract	46–53	Select exact readers and verify framework and deployment behavior
Work independently	54–59	Handle network changes, research, capstones, and continued learning

Three reader routes

A new learner should read in order, solve the three checks before the answers, and begin the lab after chapter 25. Repeat the protocol predictions after the first lab session.

A developer can start with chapters 1–12 as a diagnostic, then read 13–24 and 46–53. Use 35 and 40–45 before making action or architecture decisions. Complete the capstone rather than stopping when an error disappears.

An assessor should read 4–12, 13–24, and 25–36 in sequence. Use the platform chapters for the actual environment, then produce a complete report and a well-reasoned non-finding.

Stop-and-explain gates

Before chapter 13, explain why an unreadable response does not mean no request was sent. Before chapter 25, predict a credentialed wildcard, a public anonymous wildcard, and a denied preflight. Before chapter 46, explain how a valid reflected grant can still be unsafe. Before the capstone, reproduce a failure and its legitimate positive control.

Move on when you can explain the reason, not merely repeat the result. The mastery tracker stores those demonstrations and your unresolved questions.

I. UNDERSTAND THE BROWSER

01. The web is a system of messages

Learn to: Identify a browser, server, URL, DNS lookup, and encrypted connection.

Definition: The web is a collection of programs that exchange messages. A browser requests resources; a server answers and may perform work.

Think of it this way: A delivery service carries orders between a customer and a shop. Protecting the delivery route does not decide who may read the shop records.

Figure 01. Browser decision trace

1 **Sees**	An HTTPS resource URL and a requesting page.
2 **Evaluates**	Name resolution, connection, certificate, then message handling.
3 **Accepts as evidence**	A valid certificate for the expected host.
4 **Does not infer**	Transport security does not grant account permission.
5 **Decides**	Continue the request if transport checks succeed.
6 **Reason**	Network reachability and application sharing are different gates.

Start by separating transport, application authority, and browser response access.

Start with an address

Enter `https://ledger.example/account` in a browser. The URL is the resource address. Its scheme is https. Its host is ledger.example. Its path is /account. The internet is the network that connects computers. DNS helps resolve a host name to network addresses. A port is a number identifying a service at an address. HTTPS normally uses port 443 unless the URL specifies another one.

The browser is a client: a program requesting a service. The server is a program answering those requests. The application is the set of rules that decides what a request means. It may read a database, call another service, or return a file. These components can run on separate computers or within one local machine.

Follow a page load

The browser obtains the main document. That document can request images, stylesheets, fonts, and scripts. A visible page can therefore cause many messages to several hosts. Clicking a button may send another request without loading a new page. A tab is a viewing context, not an independent security identity or cookie store.

An API is an interface that programs use to request data or actions. A web page can ask an API for a balance, then display the result. A developer may intentionally put the page and API on different origins. Understanding why that communication is restricted, and how permission is granted, is the purpose of this book.

HTTPS protects the journey

HTTPS uses TLS to protect transport and authenticate the expected server through certificate checks. It does not establish that a caller may read a particular user's record. An encrypted request can still ask for an unauthorized action. A server can also send overly broad sharing permission over an otherwise secure connection.

The browser checks several boundaries in sequence. Reaching a server, possessing a valid session, and being allowed to expose a response to a page are separate events. Do not combine them into one word such as access.

A running example

Atlas Ledger is our fictional application. Amina opens its API dashboard. A trusted frontend displays a private training record through that API. A separate peer page represents an untrusted origin. In the local lab, all data is invented and all destinations are loopback addresses on the reader's computer.

Our key experiment will ask whether the peer can read a value that the API intended to share only with the trusted frontend. We will also inspect cases where a request reaches the API but its answer remains unreadable. Both outcomes matter.

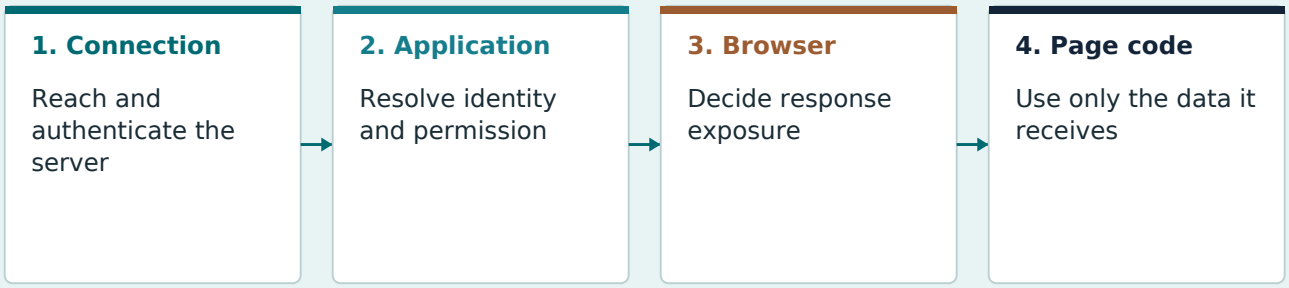
Common misconception

A successful connection does not prove successful data access. A TLS error, HTTP error, application denial, and browser read denial happen at different boundaries. Learn to name the boundary before trying to change a header.

Practice and reflection

On a page you own, open Developer Tools and select Network. Reload once. Identify the main document and one additional request. Record host, method, status, and resource type without copying cookies. Explain what happened before any application data could be displayed.

Figure 01b. Three boundaries in a page load



Summary and key takeaway

A browser interaction is a chain of decisions, not one success or failure.

Check your understanding

1.1 Does HTTPS decide which user records a page may read?

1.2 Does a new tab always create a new session?

1.3 Why inspect more than the main document request?

Answers appear in the full ebook. Sources: S01, S02

02. HTML, JavaScript, and the first API call

Learn to: Read a small form and fetch call; understand a DOM, promise, and response body.

Definition: HTML describes document structure. JavaScript can respond to events and make requests. The DOM is the browser's object representation of the document.

Think of it this way: A shop window shows the information. The order form collects values. A worker behind the counter sends the request and handles the answer.

Figure 02. Browser decision trace

1 **Sees**	A page script calls fetch with a resolved URL.
2 **Evaluates**	Request mode, destination, credentials mode, and applicable policies.
3 **Accepts as evidence**	Only a response that passes the relevant access checks.
4 **Does not infer**	A successful promise does not mean HTTP status 200 or valid JSON.
5 **Decides**	Expose a response object or reject the fetch operation.
6 **Reason**	Transport, HTTP status, body parsing, and rendering are different steps.

Trace each asynchronous step so an error at one step is not blamed on another.

Read the page before the security rule

An HTML element is a labelled piece of a document. A paragraph displays text. An input collects a value. A button can trigger an action. CSS controls appearance; it does not grant permission to read another origin. JavaScript adds behavior. The DOM lets code locate an element and change its displayed content.

```
<p id="result">No record loaded.</p>
<button id="load" type="button">Load training record</button>
```

The id gives the element a name within the document. The button's type avoids accidental form submission. A real form can submit named fields without JavaScript. Later we will compare forms with script requests because they do not have identical capabilities.

Read one asynchronous operation

```
async function loadRecord() {
  const response = await fetch('/api/training-record');
  if (!response.ok) throw new Error('Request was not successful');
  const record = await response.json();
  document.getElementById('result').textContent = record.label;
}
document.getElementById('load').addEventListener('click', loadRecord);
```

This is a reading example, not a complete secure application. A function is a named block of work. const stores a value under a name. fetch starts a request and returns a promise: a value representing future completion or failure. await waits for that result within an async function without turning the whole browser into a frozen page.

The response object contains status and allowed metadata. response.json reads and parses the body as JSON, a structured text format. Reading a body is another asynchronous step. A request can succeed at HTTP level and still fail JSON parsing if the body contains an HTML error page.

HTTP failure is not a rejected fetch

An HTTP 404 or 500 can produce a normal response object when browser access is permitted. Check response.ok or status. A network or CORS failure usually rejects the promise instead. Script often cannot distinguish the precise network-policy reason from the exception alone; Developer Tools provide more detail.

The code uses textContent to display a value as text. Putting an untrusted string into raw HTML creates a different rendering risk. Safe text display does not prove the API sharing policy is correct, and correct sharing does not make arbitrary HTML safe.

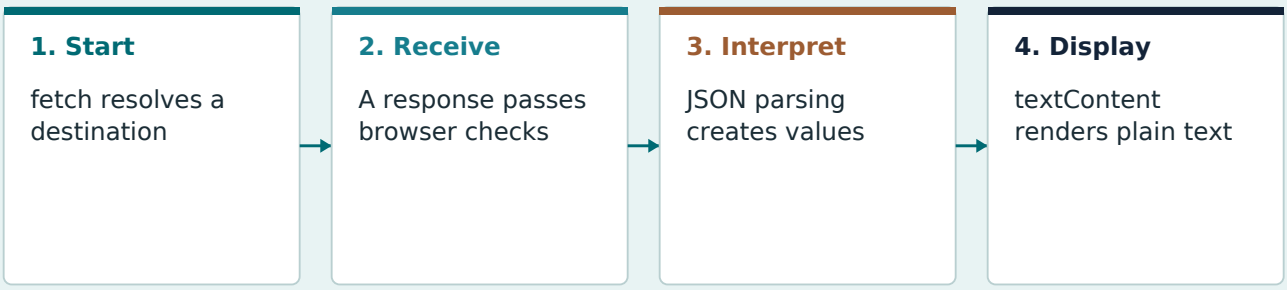
Relative and absolute destinations

The path /api/training-record is relative to the current origin. An absolute URL includes a scheme and host. The same helper can become unsafe if untrusted input selects an arbitrary destination and the helper attaches credentials or secrets. Constrain the operation before adding authority.

Practice and reflection

Explain each line without memorizing it. Identify where the request starts, where status is checked, where the body is parsed, and where text reaches the DOM. In a local page, compare a deliberate button request with automatic loading. Ask who selected the destination and which code receives the answer.

Figure 02b. Trace the code through four stages



Summary and key takeaway

A readable response, a successful HTTP result, and safely displayed data are separate achievements.

Check your understanding

2.1 Does fetch reject automatically on every HTTP 404?

2.2 What does await response.json add?

2.3 Does textContent repair a CORS misconfiguration?

Answers appear in the full ebook. Sources: S03, S04, S17

03. HTTP requests, representations, and effects

Learn to: Read methods, headers, bodies, and status; distinguish safe operations from repeatable ones.

Definition: HTTP defines message semantics. A request names an operation and a target; a response reports a result and may carry data.

Think of it this way: An envelope, its contents, and the action taken after delivery are three separate observations.

Figure 03. Browser decision trace

1 **Sees**	A method, target, request headers, and optional body.
2 **Evaluates**	Dispatch rules and then response access policy.
3 **Accepts as evidence**	The server response fields only for their defined roles.
4 **Does not infer**	An HTTP success is not proof of permission or human approval.
5 **Decides**	Send, withhold, expose, or reject according to separate rules.
6 **Reason**	Method semantics and browser read permission solve different problems.

Record dispatch, HTTP result, browser readability, and business effect independently.

Read the message

```
GET /api/private/record HTTP/1.1
Host: api.example
Origin: https://app.example
Cookie: sid=demonstration-value
```

This readable HTTP/1.1 view helps identify the method, path, headers, and credential carrier. HTTP/2 and HTTP/3 use different framing but retain the relevant concepts. Headers carry metadata. A body carries a representation such as JSON or form data. Header names are case-insensitive, but individual values follow their own syntax rules.

A response can contain a status, headers, and a body. Status 200 means an HTTP success, not necessarily authorized business access. Status 401 commonly indicates missing or invalid authentication; 403 indicates refusal; 405 rejects a method; 415 rejects a representation type. Applications can use these imperfectly. Inspect the actual result.

Safe and idempotent mean different things

A safe method has essentially read-only requested semantics. GET should not change a recovery address or approve a transfer. Idempotent means repeating the same request has the same intended effect as sending it once. DELETE can be idempotent while still deleting important data. POST is not automatically secure because it is an unsafe method.

CORS can deny script access after a safelisted request has reached the handler. If that handler changes state, the denied read does not undo the write. This is why CORS and request-forgery defenses must be understood together without treating them as identical controls.

Representations influence dispatch

Content-Type tells the recipient how the body is represented. Ordinary forms often use application/x-www-form-urlencoded or multipart/form-data. APIs often use application/json. The browser's preflight decision depends partly on these details. A server that accepts JSON text under text/plain can change the expected browser path.

The parser converts bytes into values. If a security layer reads one interpretation while the handler uses another, the check may protect the wrong operation. Keep singleton fields unambiguous and define accepted methods and types explicitly.

Worked example: four observations

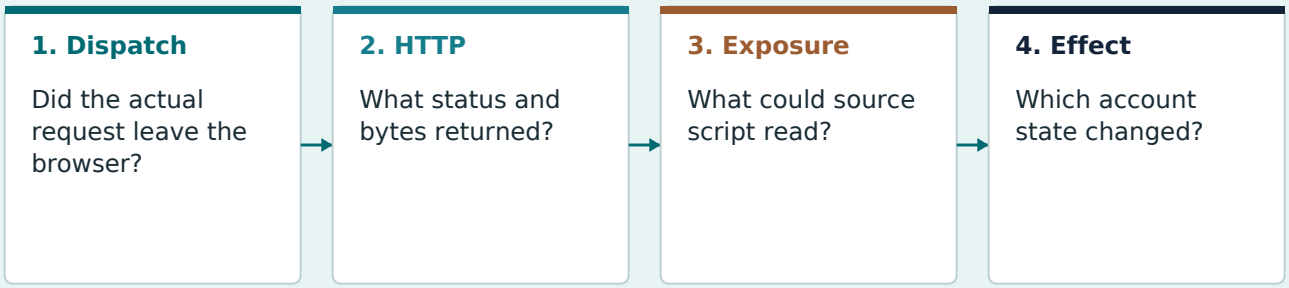
The peer submits a display-name form. Network shows a POST and a 200 response. The peer script cannot read the response. Amina's name nevertheless changes on the legitimate dashboard. The request was dispatched, the server accepted an authenticated effect, and the browser withheld the answer from the source. Calling the entire event a blocked request would hide the failure.

For a private-data read, the relevant impact is different: the peer must obtain sensitive response content under the victim context. A status code visible in a proxy capture is not enough to establish that script obtained it.

Practice and reflection

Write contracts for one public read, one private read, and one mutation. State authentication, method, media type, accepted readers, and effect. For each, explain what a 200 response can and cannot prove. Keep transport evidence, HTTP outcome, browser exposure, and business effect in separate notebook columns.

Figure 03b. Four observations for one request



Summary and key takeaway

CORS response denial does not reverse an effect the application has already accepted.

Check your understanding

3.1 Is an idempotent method necessarily read-only?

3.2 Does a proxy showing a 200 private response prove a CORS disclosure?

3.3 Why does media type matter?

Answers appear in the full ebook. Sources: S02, S22, S39

Sources and further reading

Research cutoff: 25 October 2026. Source aliases are grouped. The catalog includes direct sources and targeted further reading. Current documentation can change; consult release-specific material and errata. Historical sources are not a current browser compatibility table.

S01 - MDN: How the web works

Foundation and further reading on browsers, servers, DNS, and HTTP.

https://developer.mozilla.org/en-US/docs/Learn_web_development/Getting_started/Web_standards/How_the_web_works

S02 - RFC 9110: HTTP Semantics

Methods, status codes, representations, and HTTP semantics. CORS processing itself is defined by Fetch.

https://www.rfc-editor.org/rfc/rfc9110.html

S03 - WHATWG HTML: Forms

Browser form construction and submission. Living standard; consult the current relevant algorithm.

https://html.spec.whatwg.org/multipage/forms.html

S04 - MDN: async function

Prerequisite for reading the book's asynchronous JavaScript examples.

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/async_function

S17 - MDN: Fetch API

API model, responses, and related references. Distinguish HTTP errors from network-style rejection.

https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API

S22 - WHATWG Fetch Living Standard

Normative reference for current CORS algorithms, safelists, credential-mode handling, response filtering, redirects, and preflight cache. Consulted against the 19 October 2026 update. Original examples and tests in this book are independent teaching material. A local Authorization-wildcard implementation difference is explicitly recorded.

https://fetch.spec.whatwg.org/

S39 - OWASP: HTML5 Security Cheat Sheet

CORS and cross-document messaging review principles.

https://cheatsheetseries.owasp.org/cheatsheets/HTML5_Security_Cheat_Sheet.html